

CSS Inset Property

Complete Guide: From Basics to Advanced

CSS Guide

May 10, 2026

Contents

1	Introduction	2
1.1	What is the CSS Inset Property?	2
1.2	Evolution and Purpose	2
1.2.1	Before inset	2
1.2.2	With inset	3
1.3	Browser Support	3
2	Core Concepts	4
2.1	Understanding Inset Syntax	4
2.1.1	Syntax Overview	4
2.1.2	Value Precedence	4
2.2	Supported Values	4
2.2.1	Valid Value Types	4
2.2.2	Examples	5
2.3	Positioning Requirements	5
3	Practical Examples	7
3.1	Example 1: Full-Screen Overlay	7
3.2	Example 2: Sticky Header	8
3.3	Example 3: Positioned Card	8
3.4	Example 4: Responsive Modal	9
3.5	Example 5: Tooltip Positioning	10
4	Advanced Techniques	12
4.1	Logical Properties	12
4.1.1	Inset-Block and Inset-Inline	12
4.2	With Flex and Grid	12
4.3	With CSS Variables	13
5	Comparison with Traditional Approach	14
5.1	Code Comparison	14
5.2	Performance	14
6	Browser Compatibility & Fallback	15
6.1	Modern vs Legacy Approach	15
6.2	Progressive Enhancement	15

7	Best Practices	17
7.1	Key Recommendations	17
7.1.1	1. Use inset for Positioned Elements	17
7.1.2	2. Combine with Transform for Centering	17
7.1.3	3. Use Logical Properties for I18n	18
7.1.4	4. Provide Fallback for Older Browsers	18
7.2	Common Patterns	18
8	Common Issues & Solutions	19
8.1	Issue 1: inset Not Working	19
8.2	Issue 2: Conflicting with Margin	19
8.3	Issue 3: z-index Not Working	20
9	Summary & Quick Reference	21
9.1	When to Use inset	21
9.2	Quick Reference	21
9.2.1	inset Values	21
9.2.2	Complete Examples	21

Chapter 1

Introduction

1.1 What is the CSS Inset Property?

The `inset` property is a CSS shorthand that simultaneously sets the values for `top`, `right`, `bottom`, and `left` properties. It's used to position absolutely or fixed positioned elements relative to their nearest positioned ancestor or the viewport.

```
/* Basic syntax */
inset: 20px;
inset: 10px 20px;
inset: 10px 20px 30px;
inset: 10px 20px 30px 40px;

/* Practical example */
.modal {
  position: fixed;
  inset: 0; /* Fill entire viewport */
}
```

The `inset` property is a modern, cleaner alternative to writing separate `top`, `right`, `bottom`, and `left` properties, significantly reducing code verbosity and improving maintainability.

1.2 Evolution and Purpose

1.2.1 Before inset

```
/* Old way: Four separate properties */
.overlay {
  position: fixed;
  top: 0;
  right: 0;
  bottom: 0;
  left: 0;
}

.modal {
  position: absolute;
  top: 50px;
```

```
right: 50px;
bottom: 50px;
left: 50px;
}
```

1.2.2 With inset

```
/* Clean and concise */
.overlay {
  position: fixed;
  inset: 0;
}

.modal {
  position: absolute;
  inset: 50px;
}
```

1.3 Browser Support

The `inset` property is well-supported in modern browsers:

Browser	Version	Status
Chrome/Edge	v87+	Full Support
Firefox	v66+	Full Support
Safari	v14.1+	Full Support
Opera	v73+	Full Support
IE 11	N/A	Not Supported

Chapter 2

Core Concepts

2.1 Understanding Inset Syntax

2.1.1 Syntax Overview

```
/* Single value: all sides */
inset: 20px;

/* Two values: vertical, horizontal */
inset: 10px 20px;

/* Three values: top, horizontal, bottom */
inset: 10px 20px 30px;

/* Four values: top, right, bottom, left */
inset: 10px 20px 30px 40px;
```

2.1.2 Value Precedence

Syntax	Assigns to
<code>inset: 20px</code>	All four sides = 20px
<code>inset: 10px 20px</code>	T/B = 10px, L/R = 20px
<code>inset: 10px 20px 30px</code>	T = 10px, L/R = 20px, B = 30px
<code>inset: 10px 20px 30px 40px</code>	T = 10px, R = 20px, B = 30px, L = 40px

2.2 Supported Values

2.2.1 Valid Value Types

- **Pixel values:** 20px, 50px
- **Percentage:** 10%, 50%
- **Length units:** 2rem, 1em, 5cm
- **Negative values:** -10px

- **Auto:** auto (depends on positioning)
- **Logical values:** inset-inline, inset-block

2.2.2 Examples

```
/* Pixel units */
inset: 20px;

/* Percentage */
inset: 10%;

/* Mixed units */
inset: 10px 2rem;

/* Negative values */
inset: -5px 10px;

/* With auto */
inset: auto 20px;

/* With CSS variables */
inset: var(--spacing);
```

2.3 Positioning Requirements

The `inset` property only works with positioned elements:

```
/* Must have position property */
.fixed-overlay {
  position: fixed;
  inset: 0; /* Works */
}

.absolute-box {
  position: absolute;
  inset: 20px; /* Works */
}

.relative-box {
  position: relative;
  inset: 10px; /* Works */
}

.static-element {
  position: static;
  inset: 20px; /* Does NOT work */
}

.sticky-element {
  position: sticky;
  inset: 10px; /* Works (in some contexts) */
}
```

```
}
```


Chapter 3

Practical Examples

3.1 Example 1: Full-Screen Overlay

```
/* Modal overlay covering entire screen */
.overlay {
  position: fixed;
  inset: 0;
  background: rgba(0, 0, 0, 0.5);
  display: flex;
  align-items: center;
  justify-content: center;
}

/* Modal dialog centered */
.modal {
  position: absolute;
  inset: auto; /* Center both ways */
  top: 50%;
  left: 50%;
  transform: translate(-50%, -50%);
  width: 500px;
  height: 300px;
  background: white;
  border-radius: 8px;
  box-shadow: 0 10px 40px rgba(0, 0, 0, 0.3);
}

/* Alternative: Fill modal with padding */
.modal-alt {
  position: absolute;
  inset: 50px;
  background: white;
  border-radius: 8px;
  display: flex;
  flex-direction: column;
}

.modal-alt header {
  padding: 20px;
```

```
border-bottom: 1px solid #eee;
}

.modal-alt .content {
  flex: 1;
  padding: 20px;
  overflow-y: auto;
}
```

3.2 Example 2: Sticky Header

```
/* Sticky header using inset */
header {
  position: sticky;
  inset-block-start: 0; /* Logical property */
  background: white;
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
  z-index: 100;
}

/* Traditional approach with inset */
header {
  position: sticky;
  top: 0;
  inset: 0; /* Not typical for sticky */
}

/* Correct sticky usage */
header {
  position: sticky;
  inset: 0 auto auto 0; /* Top and left only */
  background: white;
}
```

3.3 Example 3: Positioned Card

```
/* Card with specific inset */
.card {
  position: absolute;
  inset: 20px 30px 20px 30px;
  background: white;
  border-radius: 8px;
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.15);
  padding: 20px;
}

/* Card with uniform inset */
.card-uniform {
  position: absolute;
  inset: 20px;
}
```

```
    background: white;
    border-radius: 8px;
}

/* Card positioned from right */
.card-right {
    position: fixed;
    inset: 20px 20px auto auto;
    width: 300px;
    height: 400px;
    background: white;
}
```

3.4 Example 4: Responsive Modal

```
/* Desktop: moderate inset */
.modal {
    position: fixed;
    inset: 50px;
    background: white;
    border-radius: 8px;
    box-shadow: 0 20px 60px rgba(0, 0, 0, 0.3);
    max-width: 800px;
    max-height: 600px;
    overflow-y: auto;
}

/* Tablet: more compact */
@media (max-width: 1024px) {
    .modal {
        inset: 40px;
    }
}

/* Mobile: full screen with safe area */
@media (max-width: 768px) {
    .modal {
        inset: 0;
        border-radius: 0;
    }
}

/* Account for notch on mobile */
.modal {
    inset: max(
        0px,
        env(safe-area-inset-top)
    )
    max(
        0px,
        env(safe-area-inset-right)
    )
    max(
```

```
        0px,  
        env(safe-area-inset-bottom)  
    )  
    max(  
        0px,  
        env(safe-area-inset-left)  
    );  
}  
}
```

3.5 Example 5: Tooltip Positioning

```
/* Tooltip with auto positioning */  
.tooltip {  
    position: absolute;  
    inset: auto;  
    background: #333;  
    color: white;  
    padding: 8px 12px;  
    border-radius: 4px;  
    font-size: 12px;  
    white-space: nowrap;  
    pointer-events: none;  
    z-index: 1000;  
}  
  
/* Bottom tooltip */  
.tooltip.bottom {  
    bottom: -35px;  
    left: 50%;  
    transform: translateX(-50%);  
}  
  
/* Top tooltip */  
.tooltip.top {  
    top: -35px;  
    left: 50%;  
    transform: translateX(-50%);  
}  
  
/* Right tooltip */  
.tooltip.right {  
    right: -50px;  
    top: 50%;  
    transform: translateY(-50%);  
}  
  
/* Left tooltip */  
.tooltip.left {  
    left: -50px;  
    top: 50%;  
    transform: translateY(-50%);  
}
```

```
}  
  
/* Using inset for cleaner code */  
.tooltip-modern {  
    position: absolute;  
    inset: auto auto 100% 50%;  
    transform: translateX(-50%);  
    background: #333;  
    color: white;  
    padding: 8px 12px;  
    border-radius: 4px;  
}
```

Chapter 4

Advanced Techniques

4.1 Logical Properties

4.1.1 Inset-Block and Inset-Inline

```
/* Traditional inset */
.element {
  position: absolute;
  inset: 10px;
}

/* Logical properties */
.element-logical {
  position: absolute;
  inset-block: 10px;    /* top and bottom */
  inset-inline: 20px;   /* left and right */
}

/* Equivalent to: inset: 10px 20px */

/* Individual logical properties */
.element-detailed {
  position: absolute;
  inset-block-start: 10px; /* top */
  inset-block-end: 20px;   /* bottom */
  inset-inline-start: 15px; /* left */
  inset-inline-end: 25px;  /* right */
}
```

4.2 With Flex and Grid

```
/* Flex container with positioned child */
.flex-parent {
  display: flex;
  position: relative;
  width: 300px;
  height: 300px;
```

```
}

.positioned-child {
  position: absolute;
  inset: 20px;
  background: #667eea;
}

/* Grid with overlay */
.grid-parent {
  display: grid;
  position: relative;
  grid-template-columns: repeat(3, 1fr);
  gap: 10px;
}

.grid-overlay {
  position: absolute;
  inset: 0;
  background: rgba(0, 0, 0, 0.1);
  border-radius: 8px;
}
```

4.3 With CSS Variables

```
/* Define spacing variables */
:root {
  --spacing-xs: 8px;
  --spacing-sm: 12px;
  --spacing-md: 20px;
  --spacing-lg: 30px;
}

/* Use with inset */
.dialog {
  position: fixed;
  inset: var(--spacing-lg);
  background: white;
  border-radius: 8px;
}

.card {
  position: absolute;
  inset: var(--spacing-md) var(--spacing-lg);
}

/* Responsive variables */
@media (max-width: 768px) {
  :root {
    --spacing-lg: 10px;
  }
}
```

Chapter 5

Comparison with Traditional Approach

5.1 Code Comparison

Pattern	Traditional	With inset
Full screen overlay	4 properties	1 property
Uniform spacing	4 properties	1 property
Custom values	4 properties	1 property
Readability	More verbose	Cleaner
Maintenance	More lines	Fewer lines

5.2 Performance

No performance difference: The `inset` property compiles to the same output as individual `top/right/bottom/left` properties, so there's no performance penalty.

Chapter 6

Browser Compatibility & Fallback

6.1 Modern vs Legacy Approach

```
/* Modern browsers (Chrome 87+, Firefox 66+, Safari 14.1+) */
.overlay {
  position: fixed;
  inset: 0;
}

/* With fallback for older browsers */
.overlay {
  position: fixed;
  top: 0;
  right: 0;
  bottom: 0;
  left: 0;
  /* inset: 0; */ /* Commented out for compatibility */
}

/* Using @supports for feature detection */
@supports (inset: 0) {
  .overlay {
    inset: 0;
  }
}
```

6.2 Progressive Enhancement

```
/* Default: works everywhere */
.modal {
  position: fixed;
  top: 50px;
  right: 50px;
  bottom: 50px;
  left: 50px;
}
```

```
/* Enhanced: modern browsers */
@supports (inset: 0) {
  .modal {
    inset: 50px;
  }
}
```

Chapter 7

Best Practices

7.1 Key Recommendations

7.1.1 1. Use inset for Positioned Elements

```
/* Good: With position */
.overlay {
  position: fixed;
  inset: 0;
}

/* Wrong: No position property */
.element {
  inset: 20px; /* Won't work */
}
```

7.1.2 2. Combine with Transform for Centering

```
/* Center an element */
.centered {
  position: absolute;
  inset: auto;
  top: 50%;
  left: 50%;
  transform: translate(-50%, -50%);
}

/* Or use modern approach */
.centered-modern {
  position: absolute;
  inset: 0;
  display: flex;
  align-items: center;
  justify-content: center;
}
```

7.1.3 3. Use Logical Properties for I18n

```
/* For RTL languages use logical properties */
.sidebar {
  position: absolute;
  inset-block: 0;
  inset-inline-start: 0;
  width: 250px;
}

/* Automatically flips for RTL */
[dir="rtl"] .sidebar {
  inset-inline-end: 0;
  inset-inline-start: auto;
}
```

7.1.4 4. Provide Fallback for Older Browsers

```
.overlay {
  position: fixed;
  top: 0;
  right: 0;
  bottom: 0;
  left: 0;
}

@supports (inset: 0) {
  .overlay {
    inset: 0;
  }
}
```

7.2 Common Patterns

Pattern	inset Value
Full screen overlay	<code>inset: 0</code>
With uniform margin	<code>inset: 20px</code>
Vertical + horizontal	<code>inset: 10px 20px</code>
Custom all sides	<code>inset: 10px 20px 30px 40px</code>
Auto center	<code>inset: auto</code>

Chapter 8

Common Issues & Solutions

8.1 Issue 1: inset Not Working

Problem: Element doesn't move with inset values.

Cause: Missing or wrong position property.

Solution:

```
/* Wrong: No position */
.element {
    inset: 20px; /* Doesn't work */
}

/* Correct: Add position */
.element {
    position: absolute;
    inset: 20px; /* Works */
}
```

8.2 Issue 2: Conflicting with Margin

Problem: Margins interfere with inset positioning.

Solution:

```
/* Clear conflicting properties */
.overlay {
    position: fixed;
    inset: 0;
    margin: 0; /* Reset margins */
}

/* Use padding instead if needed */
.container {
    position: absolute;
    inset: 50px;
    padding: 20px;
}
```

8.3 Issue 3: z-index Not Working

Problem: Element appears behind others despite inset.

Solution:

```
/* Add z-index for layering */
.overlay {
  position: fixed;
  inset: 0;
  z-index: 999;
  background: rgba(0, 0, 0, 0.5);
}

/* Ensure stacking context */
.modal {
  position: fixed;
  inset: 50px;
  z-index: 1000;
}
```

Chapter 9

Summary & Quick Reference

9.1 When to Use inset

Use `inset` when:

- Positioning with fixed or absolute
- Creating overlays and modals
- Need multiple offset values
- Building responsive layouts
- Supporting modern browsers

9.2 Quick Reference

9.2.1 inset Values

```
/* Full screen: covers entire viewport */
inset: 0;

/* Uniform spacing from edges */
inset: 20px;

/* Vertical and horizontal */
inset: 10px 20px;

/* All four sides */
inset: 10px 20px 30px 40px;

/* Mixed units */
inset: 10px 2rem 15px auto;
```

9.2.2 Complete Examples

```
/* Full-screen modal */
.modal {
  position: fixed;
  inset: 0;
}

/* Card with padding */
.card {
  position: absolute;
  inset: 20px;
}

/* Right sidebar */
.sidebar {
  position: fixed;
  inset: 0 0 0 auto;
  width: 300px;
}

/* Top sticky bar */
.header {
  position: sticky;
  inset: 0 0 auto 0;
}
```

Thank you for reading!

For more information, visit [MDN Web Docs](#) and [web.dev](#)